

JAVA PROGRAMMING LANGUAGE

Handwritten Notes — Complete Masterclass Edition (117 Pages)

TABLE OF CONTENTS / INDEX (13 MODULES)

1. History of Java & Core Features (Pages 1 – 5)
2. JVM, JRE, JDK & Program Generations (Pages 6 – 9)
3. Input Handling: Scanner, BigInteger & BigDecimal (Pages 10 – 14)
4. Java Expressions, Data Types & Flow Control (Pages 15 – 18)
5. Control Flow Statements (If-Else, Switch, Loops) (Pages 19 – 23)
6. Classes, Objects & Constructors (Pages 24 – 32)
7. Object-Oriented Programming (OOPs) Concepts (Pages 33 – 42)
8. Inheritance, Super Keyword & Method Overriding (Pages 43 – 52)
9. Polymorphism, Abstraction & Abstract Classes (Pages 53 – 62)
10. Interfaces, Default/Static Methods & Packages (Pages 63 – 72)
11. Exception Handling (Try-Catch, Custom Exceptions) (Pages 73 – 85)
12. String Handling, StringBuilder & StringBuffer (Pages 86 – 100)
13. Enums, Static Keyword & JVM Memory Model (Pages 101 – 117)

1. History of Java & Core Features

- Initiated in June 1991 by James Gosling, Mike Sheridan, and Patrick Naughton (Green Team).
- Initially named Greentalk (.gt) and Oak, later renamed Java in 1995.
- JDK 1.0 released on January 23, 1996.
- Core Attributes: Simple, Portable, Secure, High-Performance, Multithreaded, Platform-Independent, Robust, Object-Oriented, Architecture-Neutral, Dynamic.

2. Input Handling with Scanner & BigInteger

```
import java.util.Scanner;
import java.math.BigInteger;

public class InputDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter large number: ");
        BigInteger b1 = sc.nextBigInteger();
        BigInteger b2 = BigInteger.valueOf(100);
        System.out.println("Result: " + b1.add(b2));
    }
}
```

3. Object-Oriented Concepts, Inheritance & Exception Handling

```
// Inheritance, super keyword, and Custom Exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String msg) { super(msg); }
}

class Person {
    int age;
    Person(int age) { this.age = age; }
}

class Student extends Person {
    Student(int age) throws InvalidAgeException {
        super(age); // Call superclass constructor
        if (age < 18) throw new InvalidAgeException("Underage student!");
    }
}
```

4. JVM Memory Model Areas

- Metaspace: Stores class metadata, bytecodes, static variables.
- Heap: Stores all instantiated objects and instance variables.
- Stack: Holds stack frames, method calls, local variables & object references.
- PC Register: Holds the address of the currently executing JVM instruction per thread.
- Native Method Stack: Stores native (C/C++) method execution state via JNI.